```
# ============================================
# 1. CONFIGURACIÓN DEL ENTORNO (GOOGLE COLAB)
# ============================================

!pip install -q streamlit pyngrok pandas numpy scikit-learn plotly
!pip install -q streamlit-plotly-events
# Permite ejecutar Streamlit dentro de Colab
from pyngrok import ngrok
```

```
# ============================================
# 2. IMPORTACIÓN DE LIBRERÍAS
# ============================================

import pandas as pd
import numpy as np
import streamlit as st
import plotly.express as px

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Aumentar el límite de Pandas Styler para permitir visualizar bases de datos grandes
pd.set_option("styler.render.max_elements", 5000000)
```

```
# ============================================
# 3. CARGA DINÁMICA DEL DATASET
# ============================================

def load_dataset(uploaded_file):
    '''
    Carga el dataset de forma dinámica.
    Permite reutilizar el sistema con diferentes fuentes de datos.
    '''
    if uploaded_file.name.endswith('.csv'):
        try:
            # Intento 1: Leer con separador estándar (coma)
            return pd.read_csv(uploaded_file, low_memory=False)
        except Exception:
            # Intento 2: Si el archivo está separado por ';' o tiene líneas corruptas
            uploaded_file.seek(0) # Reinicia el puntero del archivo
            return pd.read_csv(uploaded_file, sep=';', on_bad_lines='skip', low_memory=False)
    elif uploaded_file.name.endswith('.xlsx'):
        return pd.read_excel(uploaded_file)
    else:
        st.error("Formato no soportado")
```

```
        return None
```

```
# ============================================
# 4. CÁLCULO DE VARIABLES RFM
# ============================================

def calculate_rfm(df, customer_col, date_col, amount_col):
    df[date_col] = pd.to_datetime(df[date_col], dayfirst=True, format='mixed', errors='coerce')
    df[amount_col] = pd.to_numeric(df[amount_col], errors='coerce')

    # MITIGACIÓN RIESGO: Filtrar montos menores o iguales a cero previos al logaritmo
    df = df.dropna(subset=[amount_col, date_col])
    df = df[df[amount_col] > 0]

    reference_date = df[date_col].max() + pd.Timedelta(days=1)

    rfm = df.groupby(customer_col).agg({
        date_col: lambda x: (reference_date - x.max()).days,
        customer_col: 'count',
        amount_col: 'sum'
    })
    rfm.columns = ['Recency', 'Frequency', 'Monetary']
    return rfm.reset_index()
```

```
# ============================================
# 5. NORMALIZACIÓN Y MÉTRICAS DE VALIDACIÓN
# ============================================

def scale_features(rfm_df):
    # Corrección Crítica: Transformación Logarítmica para mitigar Outliers
    # Comprime la brecha entre valores atípicos y promedios antes del escalamiento
    rfm_log = rfm_df[['Recency', 'Frequency', 'Monetary']].copy()

    for col in rfm_log.columns:
        # np.maximum evita errores con valores negativos o ceros (log(0) es indefinido)
        rfm_log[col] = np.log1p(np.maximum(rfm_log[col], 0))

    scaler = StandardScaler()
    scaled = scaler.fit_transform(rfm_log)
    return scaled

def calculate_elbow_silhouette(data, max_k=10):
    inertia = []
    silhouette = []
    db_scores = []
    business_scores = []
    for k in range(2, max_k+1):
        model = KMeans(n_clusters=k, random_state=42)
        labels = model.fit_predict(data)

        sil_score = silhouette_score(data, labels)
        inertia.append(model.inertia_)
        silhouette.append(sil_score)
        db_scores.append(davies_bouldin_score(data, labels))
        penalty = 0.08 * abs(k - 3)
```

```
        business_scores.append(sil_score - penalty)

    return inertia, silhouette, db_scores, business_scores
```

```
# ============================================
# 6. ENTRENAMIENTO DEL MODELO K-MEANS (K)
# ============================================

def train_kmeans(data, k):
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    labels = kmeans.fit_predict(data)
    return kmeans, labels
```

```
# ============================================
# 7. ASIGNACIÓN DE SEGMENTOS RFM (AJUSTADA)
# ============================================

def assign_dynamic_segments(rfm):
    '''
    Mapeo dinámico: Evalúa matemáticamente los centroides descubiertos.
    Score = Monetary_z + Frequency_z - Recency_z.
    '''
   # Uso logaritmo interno para evaluar los centroides sin distorsión
    rfm_log = rfm.copy()
    rfm_log[['Recency', 'Frequency', 'Monetary']] = np.log1p(rfm[['Recency', 'Frequency', 'Monetary']])
    centroids = rfm_log.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']].mean()

    scaler = StandardScaler()
    scaled_centroids = pd.DataFrame(
        scaler.fit_transform(centroids), columns=['R_z', 'F_z', 'M_z'], index=centroids.index
    )
    scaled_centroids['Score'] = scaled_centroids['M_z'] + scaled_centroids['F_z'] - scaled_centroids['R_z']
    scaled_centroids = scaled_centroids.sort_values('Score', ascending=False)

    cluster_to_label = {}
    for idx, (cluster_id, row) in enumerate(scaled_centroids.iterrows()):
        if idx == 0: cluster_to_label[cluster_id] = 'VIP'
        elif idx == len(scaled_centroids) - 1: cluster_to_label[cluster_id] = 'Perdidos'
        else:
            if row['R_z'] > 0: cluster_to_label[cluster_id] = 'En Riesgo'
            else: cluster_to_label[cluster_id] = 'Leales'

    rfm['Segmento'] = rfm['Cluster'].map(cluster_to_label)
    return rfm
```

```
# ============================================
# 8. RECOMENDACIONES ESTRATÉGICAS DE MARKETING
# ============================================
```

```
def marketing_recommendations(segment):
    recommendations = {
        "VIP": [
            "Ofrecer programas de fidelización exclusivos",
            "Acceso anticipado a promociones y lanzamientos",
            "Comunicación personalizada 1 a 1",
            "Incentivos por referidos"
        ],
        "Leales": [
            "Descuentos progresivos por frecuencia de compra",
            "Campañas de cross-selling y up-selling",
            "Programas de puntos",
            "Encuestas de satisfacción"
        ],
        "En Riesgo": [
            "Campañas de reactivación con descuentos agresivos",
            "Recordatorios personalizados por email/SMS",
            "Ofertas por tiempo limitado",
            "Análisis de causas de abandono"
        ],
        "Perdidos": [
            "Campañas de recuperación de bajo costo",
            "Ofertas de bienvenida para reenganche",
            "Excluir de campañas costosas",
            "Evaluar eliminación del CRM activo"
        ]
    }

    return recommendations.get(segment, [])
```

```
# ============================================
# 9. DASHBOARD INTERACTIVO STREAMLIT
# ============================================

st.title("Segmentación de Clientes – Retail (RFM + K-Means)")

uploaded_file = st.file_uploader("Cargar dataset transaccional")

if uploaded_file:
    df = load_dataset(uploaded_file)

    def_cust_idx = 0
    def_date_idx = next((i for i, c in enumerate(df.columns) if 'date' in c.lower() or 'fecha' in c.lower()), 0)
    def_amt_idx = next((i for i, c in enumerate(df.columns) if 'monto' in c.lower() or 'total' in c.lower() or 'amount' in c.lower()), 0)

    col1, col2, col3 = st.columns(3)
    with col1: customer_col = st.selectbox("Columna ID Cliente", df.columns, index=def_cust_idx)
    with col2: date_col = st.selectbox("Columna Fecha", df.columns, index=def_date_idx)
    with col3: amount_col = st.selectbox("Columna Monto", df.columns, index=def_amt_idx)

    if 'ejecutado' not in st.session_state:
        st.session_state.ejecutado = False
```

```
        if st.button("Ejecutar Segmentación"):
            st.session_state.ejecutado = True

        if st.session_state.ejecutado:
            try:
                rfm = calculate_rfm(df, customer_col, date_col, amount_col)
                scaled = scale_features(rfm)

                st.markdown("---")
                st.subheader("Optimización del Número de Clusters ($K$)")

                inertia, silhouette, db_scores, business_scores = calculate_elbow_silhouette(scaled, max_k=10)
                best_k = int(np.argmax(business_scores) + 2)

                sil_df = pd.DataFrame({
                    'K': range(2, 11),
                    'Silhouette (Puro)': silhouette,
                    'Davies-Bouldin': db_scores,
                    'Score Negocio (Ajustado)': business_scores
                })

                st.line_chart(sil_df.set_index('K'))
                st.success(f"**$K$ Óptimo matemático (Ajustado por Negocio) aplicado:** {best_k}")

                model, labels = train_kmeans(scaled, best_k)
                rfm['Cluster'] = labels

                rfm = assign_dynamic_segments(rfm)

                st.markdown("---")
                col_bar, col_3d = st.columns([1, 2])

                color_map = {
                    'VIP': 'lightgreen',
                    'Perdidos': 'red',
                    'En Riesgo': 'blue',
                    'Leales': 'blue'
                }

                with col_bar:
                    st.subheader("Distribución de Segmentos")
                    segment_counts = rfm['Segmento'].value_counts().reset_index()
                    segment_counts.columns = ['Segmento', 'Cantidad']
                    fig_bar = px.bar(
                        segment_counts,
                        x='Segmento',
                        y='Cantidad',
                        color='Segmento',
                        color_discrete_map=color_map
                    )
                    fig_bar.update_layout(showlegend=False)
                    st.plotly_chart(fig_bar)

                with col_3d:
                    st.subheader("Topología 3D")
                    fig = px.scatter_3d(
                        rfm, x='Recency', y='Frequency', z='Monetary',
```

```
                color='Segmento', opacity=0.8,
                color_discrete_map=color_map,
                hover_data={'Monetary': ':$,.2f'},
                log_x=True, log_y=True, log_z=True,
                title="Distribución RFM por Segmento (Escala Logarítmica)"
            )
            fig.update_layout(
                margin=dict(l=0, r=0, b=0, t=30),
                scene=dict(zaxis=dict(tickformat="$,.2f"))
            )
            st.plotly_chart(fig)

        # --- NUEVO BLOQUE: GRÁFICO 2D PARA EXPORTACIÓN ---
        st.markdown("---")
        st.subheader("Proyección 2D (Ideal para exportación y reportes)")

        # Normalización MinMax sobre logaritmos para replicar el eje 0-1 de la imagen
        rfm_log_2d = np.log1p(rfm[['Recency', 'Monetary']])
        rfm_norm = (rfm_log_2d - rfm_log_2d.min()) / (rfm_log_2d.max() - rfm_log_2d.min())
        rfm['Recency_Norm'] = rfm_norm['Recency']
        rfm['Monetary_Norm'] = rfm_norm['Monetary']

        fig_2d = px.scatter(
            rfm, x='Recency_Norm', y='Monetary_Norm',
            color='Segmento', opacity=0.7,
            color_discrete_map=color_map,
            hover_data={'Monetary': ':$,.2f', 'Recency': True},
            title="Topología de Clústeres (Monetary vs Recency)"
        )
        fig_2d.update_layout(
            xaxis_title="Recency Log Normalizado (Actividad Reciente)",
            yaxis_title="Monetary Log Normalizado (Gasto Histórico)"
        )
        st.plotly_chart(fig_2d)
        st.caption("La frontera matemática de decisión (Decision Boundary) revela cómo los clientes con alto gasto (eje Y) son degradados drásticamente al aumentar los días inactivos (eje
        # ----------------------------------------------

        st.markdown("---")
        st.subheader("Recomendaciones de Marketing")
        segmentos_disponibles = rfm['Segmento'].unique().tolist()
        selected_segment = st.selectbox("Selecciona un segmento descubierto", segmentos_disponibles)

        recs = marketing_recommendations(selected_segment)
        for r in recs:
            st.markdown(f"✅ {r}")

        st.markdown("---")
        st.subheader(f"Transacciones del segmento: {selected_segment}")
        clientes_segmento = rfm[rfm['Segmento'] == selected_segment][customer_col]
        df_segmento = df[df[customer_col].isin(clientes_segmento)]

        st.dataframe(df_segmento.style.format({amount_col: "${:,.2f}"}))

    except Exception as e:
        st.error(f"🚨 Error al procesar: Verifica que seleccionaste columnas válidas para Fecha y Monto. Detalle: {e}")
        st.session_state.ejecutado = False
```

```
2026-03-22 02:08:19.669 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-03-22 02:08:19.670 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-03-22 02:08:19.672 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-03-22 02:08:19.673 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-03-22 02:08:19.674 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-03-22 02:08:19.675 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-03-22 02:08:19.676 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-03-22 02:08:19.676 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-03-22 02:08:19.678 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

```
# Ejecutar Streamlit en Colab
# Obtén tu authtoken de ngrok en https://dashboard.ngrok.com/get-started/your-authtoken
# y reemplaza 'YOUR_AUTHTOKEN' con tu token real.
ngrok.set_auth_token('38CulDXlM53o6OGkHScEziK918U_4TJ3Erf2M63j3FoxhJqPX')
public_url = ngrok.connect(8501)
print(public_url)

# Guardar el código de Streamlit en un archivo app.py
with open('app.py', 'w') as f:
    f.write("""
import pandas as pd
import numpy as np
import streamlit as st
import plotly.express as px

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Aumentar el límite de Pandas Styler para permitir visualizar bases de datos grandes
pd.set_option("styler.render.max_elements", 5000000)


# ==========================================
# 3. CARGA DINÁMICA DEL DATASET
# ==========================================

def load_dataset(uploaded_file):
    '''
    Carga el dataset de forma dinámica.
    Permite reutilizar el sistema con diferentes fuentes de datos.
    '''
    if uploaded_file.name.endswith('.csv'):
        try:
            # Intento 1: Leer con separador estándar (coma)
            return pd.read_csv(uploaded_file)
        except Exception:
            # Intento 2: Si el archivo está separado por ';' o tiene líneas corruptas
            uploaded_file.seek(0) # Reinicia el puntero del archivo
            return pd.read_csv(uploaded_file, sep=';', on_bad_lines='skip')
    elif uploaded_file.name.endswith('.xlsx'):
        return pd.read_excel(uploaded_file)
    else:
```

```
        st.error("Formato no soportado")
        return None

# ==========================================
# 4. CÁLCULO DE VARIABLES RFM
# ==========================================
def calculate_rfm(df, customer_col, date_col, amount_col):
    df[date_col] = pd.to_datetime(df[date_col], dayfirst=True, format='mixed', errors='coerce')
    df[amount_col] = pd.to_numeric(df[amount_col], errors='coerce')

    # MITIGACIÓN RIESGO: Filtrar montos menores o iguales a cero previos al logaritmo
    df = df.dropna(subset=[amount_col, date_col])
    df = df[df[amount_col] > 0]

    reference_date = df[date_col].max() + pd.Timedelta(days=1)

    rfm = df.groupby(customer_col).agg({
        date_col: lambda x: (reference_date - x.max()).days,
        customer_col: 'count',
        amount_col: 'sum'
    })
    rfm.columns = ['Recency', 'Frequency', 'Monetary']
    return rfm.reset_index()

# ==========================================
# 5. NORMALIZACIÓN Y MÉTRICAS DE VALIDACIÓN
# ==========================================
def scale_features(rfm_df):
    # Corrección Crítica: Transformación Logarítmica para mitigar Outliers
    # Comprime la brecha entre valores atípicos y promedios antes del escalamiento
    rfm_log = rfm_df[['Recency', 'Frequency', 'Monetary']].copy()

    for col in rfm_log.columns:
        # np.maximum evita errores con valores negativos o ceros (log(0) es indefinido)
        rfm_log[col] = np.log1p(np.maximum(rfm_log[col], 0))

    scaler = StandardScaler()
    scaled = scaler.fit_transform(rfm_log)
    return scaled

def calculate_elbow_silhouette(data, max_k=10):
    inertia = []
    silhouette = []
    db_scores = []
    business_scores = []
    for k in range(2, max_k+1):
        model = KMeans(n_clusters=k, random_state=42)
        labels = model.fit_predict(data)

        sil_score = silhouette_score(data, labels)
        inertia.append(model.inertia_)
        silhouette.append(sil_score)
        db_scores.append(davies_bouldin_score(data, labels))
        penalty = 0.08 * abs(k - 3)
        business_scores.append(sil_score - penalty)

    return inertia, silhouette, db_scores, business_scores
```

```
# ============================================
# 6. ENTRENAMIENTO DEL MODELO K-MEANS
# ============================================
def train_kmeans(data, k):
    \"\"\"Entrena el modelo K-Means usando el K óptimo descubierto.\"\"\"
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    labels = kmeans.fit_predict(data)
    return kmeans, labels

# ============================================
# 7. ASIGNACIÓN DINÁMICA DE SEGMENTOS RFM
# ============================================
def assign_dynamic_segments(rfm):
    \"\"\"
    Mapeo dinámico: Evalúa matemáticamente los centroides descubiertos.
    Score = Monetary_z + Frequency_z - Recency_z.
    \"\"\"
    rfm_log = rfm.copy()
    rfm_log[['Recency', 'Frequency', 'Monetary']] = np.log1p(rfm[['Recency', 'Frequency', 'Monetary']])
    centroids = rfm_log.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']].mean()

    scaler = StandardScaler()
    scaled_centroids = pd.DataFrame(
        scaler.fit_transform(centroids), columns=['R_z', 'F_z', 'M_z'], index=centroids.index
    )
    scaled_centroids['Score'] = scaled_centroids['M_z'] + scaled_centroids['F_z'] - scaled_centroids['R_z']
    scaled_centroids = scaled_centroids.sort_values('Score', ascending=False)

    cluster_to_label = {}
    for idx, (cluster_id, row) in enumerate(scaled_centroids.iterrows()):
        if idx == 0: cluster_to_label[cluster_id] = 'VIP'
        elif idx == len(scaled_centroids) - 1: cluster_to_label[cluster_id] = 'Perdidos'
        else:
            if row['R_z'] > 0: cluster_to_label[cluster_id] = 'En Riesgo'
            else: cluster_to_label[cluster_id] = 'Leales'

    rfm['Segmento'] = rfm['Cluster'].map(cluster_to_label)
    return rfm


# ============================================
# 8. RECOMENDACIONES ESTRATÉGICAS DE MARKETING
# ============================================
def marketing_recommendations(segment):
    recommendations = {
        "VIP": [
            "Programas de fidelización premium",
            "Ofertas exclusivas y lanzamientos anticipados",
            "Beneficios por referidos",
            "Atención personalizada"
        ],
        "Leales": [
            "Promociones por frecuencia",
            "Cross-selling y up-selling",
            "Programa de puntos",
            "Comunicación periódica personalizada"
```

```
            ],
            "En Riesgo": [
                "Campañas de reactivación",
                "Descuentos por tiempo limitado",
                "Recordatorios personalizados",
                "Análisis de churn"
            ],
            "Perdidos": [
                "Campañas de recuperación de bajo costo",
                "Ofertas de reenganche",
                "Excluir de campañas costosas",
                "Depuración del CRM"
            ]
        }
        return recommendations.get(segment, [])

    # ============================================
    # 9. DASHBOARD INTERACTIVO STREAMLIT
    # ============================================
    st.title("Segmentación de Clientes – Retail (RFM + K-Means)")

    uploaded_file = st.file_uploader("Cargar dataset transaccional")

    if uploaded_file:
        df = load_dataset(uploaded_file)

        def_cust_idx = 0
        def_date_idx = next((i for i, c in enumerate(df.columns) if 'date' in c.lower() or 'fecha' in c.lower()), 0)
        def_amt_idx = next((i for i, c in enumerate(df.columns) if 'monto' in c.lower() or 'total' in c.lower() or 'amount' in c.lower()), 0)

        col1, col2, col3 = st.columns(3)
        with col1: customer_col = st.selectbox("Columna ID Cliente", df.columns, index=def_cust_idx)
        with col2: date_col = st.selectbox("Columna Fecha", df.columns, index=def_date_idx)
        with col3: amount_col = st.selectbox("Columna Monto", df.columns, index=def_amt_idx)

        if 'ejecutado' not in st.session_state:
            st.session_state.ejecutado = False

        if st.button("Ejecutar Segmentación"):
            st.session_state.ejecutado = True

        if st.session_state.ejecutado:
            try:
                rfm = calculate_rfm(df, customer_col, date_col, amount_col)
                scaled = scale_features(rfm)

                st.markdown("---")
                st.subheader("Optimización del Número de Clusters ($K$)")

                inertia, silhouette, db_scores, business_scores = calculate_elbow_silhouette(scaled, max_k=10)
                best_k = int(np.argmax(business_scores) + 2)

                sil_df = pd.DataFrame({
                    'K': range(2, 11),
                    'Silhouette (Puro)': silhouette,
                    'Davies-Bouldin': db_scores,
                    'Score Negocio (Ajustado)': business_scores
```

```
                })

                st.line_chart(sil_df.set_index('K'))
                st.success(f"**$K$ Óptimo matemático (Ajustado por Negocio) aplicado:** {best_k}")

                model, labels = train_kmeans(scaled, best_k)
                rfm['Cluster'] = labels

                rfm = assign_dynamic_segments(rfm)

                st.markdown("---")
                col_bar, col_3d = st.columns([1, 2])

                color_map = {
                    'VIP': 'lightgreen',
                    'Perdidos': 'red',
                    'En Riesgo': 'blue',
                    'Leales': 'blue'
                }

                with col_bar:
                    st.subheader("Distribución de Segmentos")
                    segment_counts = rfm['Segmento'].value_counts().reset_index()
                    segment_counts.columns = ['Segmento', 'Cantidad']
                    fig_bar = px.bar(
                        segment_counts,
                        x='Segmento',
                        y='Cantidad',
                        color='Segmento',
                        color_discrete_map=color_map
                    )
                    fig_bar.update_layout(showlegend=False)
                    st.plotly_chart(fig_bar)

                with col_3d:
                    st.subheader("Topología 3D")
                    fig = px.scatter_3d(
                        rfm, x='Recency', y='Frequency', z='Monetary',
                        color='Segmento', opacity=0.8,
                        color_discrete_map=color_map,
                        hover_data={'Monetary': ':$,.2f'},
                        log_x=True, log_y=True, log_z=True,
                        title="Distribución RFM por Segmento (Escala Logarítmica)"
                    )
                    fig.update_layout(
                        margin=dict(l=0, r=0, b=0, t=30),
                        scene=dict(zaxis=dict(tickformat="$,.2f"))
                    )
                    st.plotly_chart(fig)

                # --- NUEVO BLOQUE: GRÁFICO 2D PARA EXPORTACIÓN ---
                st.markdown("---")
                st.subheader("Proyección 2D (Ideal para exportación y reportes)")

                # Normalización MinMax sobre logaritmos para replicar el eje 0-1 de la imagen
                rfm_log_2d = np.log1p(rfm[['Recency', 'Monetary']])
                rfm_norm = (rfm_log_2d - rfm_log_2d.min()) / (rfm_log_2d.max() - rfm_log_2d.min())
                rfm['Recency_Norm'] = rfm_norm['Recency']
```

```
            rfm['Monetary_Norm'] = rfm_norm['Monetary']

            fig_2d = px.scatter(
                rfm, x='Recency_Norm', y='Monetary_Norm',
                color='Segmento', opacity=0.7,
                color_discrete_map=color_map,
                hover_data={'Monetary': ':$,.2f', 'Recency': True},
                title="Topología de Clústeres (Monetary vs Recency)"
            )
            fig_2d.update_layout(
                xaxis_title="Recency Log Normalizado (Actividad Reciente)",
                yaxis_title="Monetary Log Normalizado (Gasto Histórico)"
            )
            st.plotly_chart(fig_2d)
            st.caption("La frontera matemática de decisión (Decision Boundary) revela cómo los clientes con alto gasto (eje Y) son degradados drásticamente al aumentar los días inactivos (eje
            # ------------------------------------------------

            st.markdown("---")
            st.subheader("Recomendaciones de Marketing")
            segmentos_disponibles = rfm['Segmento'].unique().tolist()
            selected_segment = st.selectbox("Selecciona un segmento descubierto", segmentos_disponibles)

            recs = marketing_recommendations(selected_segment)
            for r in recs:
                st.markdown(f"✅ {r}")

            st.markdown("---")
            st.subheader(f"Transacciones del segmento: {selected_segment}")
            clientes_segmento = rfm[rfm['Segmento'] == selected_segment][customer_col]
            df_segmento = df[df[customer_col].isin(clientes_segmento)]

            st.dataframe(df_segmento.style.format({amount_col: "${:,.2f}"}))

        except Exception as e:
            st.error(f"🚨 Error al procesar: Verifica que seleccionaste columnas válidas para Fecha y Monto. Detalle: {e}")
            st.session_state.ejecutado = False
""")

# Ahora, ejecuta el archivo app.py con streamlit
!streamlit run app.py
```

```
NgrokTunnel: "https://overcoolly-cycadaceous-colten.ngrok-free.dev" -> "http://localhost:8501"

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.


  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://172.28.0.12:8501
  External URL: http://35.194.239.203:8501

/content/app.py:27: DtypeWarning: Columns (4) have mixed types. Specify dtype option on import or set low_memory=False.
  return pd.read_csv(uploaded_file)
/content/app.py:27: DtypeWarning: Columns (4) have mixed types. Specify dtype option on import or set low_memory=False.
  return pd.read_csv(uploaded_file)
```